



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)
FACULTY OF INFORMATION TECHNOLOGY

Data Structures Section D20 Session: Spring 2026

Assignment no **Six**:

🧐 **DON'T PANIC** 🧐

Due on Portal Only

Submission in Two Parts on Portal Only

Part One is due **WED 11:59 PM June 17, 2026**

Part Two is due **Mon 11:59 PM June 22, 2026**

The Use of ChatGPT is Allowed for Understanding

Copy (Even One line) from ChatGPT or Anyone or Anywhere Else is Zero Marks

50 + 50 + 20 (Bonus) Marks

Final Marks are based on **Assignment Evaluation**

You Should be Able to Write the Submitted Code in Evaluation

Topic	Recursion
Objective/ Outcome	• Applications of Heap • Applications of AVLTree • Hashtable and Applications
3 Tasks	Heap: 1 Task AVLTree: 1 Task Hashtable: 1 Task

CLO	Description	Taxonomy Level	PLO
2	Solve basic problems using fundamental concepts of programming language (C++).	C3 (Apply)	2

General Instructions

- 🧐 **DON'T PANIC** 🧐
- Make sure to read the **whole assignment document** carefully and take **notes in your DS Notebook**
- **DON'T DISREGARD INSTRUCTIONS**
- Recommended to submit work at least one hour before the deadline to avoid any internet or portal issues



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)
FACULTY OF INFORMATION TECHNOLOGY

- **CORRECTION** of any FOUND ERRORS or MISTAKES is part of the assignment and Result in Bonus Marks

DON'T ATTEMPT THE SOLUTION BEFORE CAREFUL READING OF THE ASSIGNMENT DESCRIPTION (and taking notes in the DS notebook)

(Your DS Notebook will be checked for Assignment 6 Evaluation, **No Notes No Marks**)

MARKS ARE BASED ON EFFORT AND NOT CORRECTION

- **Cheating or Copy is Cheating Yourself out of Learning**
- **Cheating or Copy may give you undeserved marks in the assignment but you lose in quizzes, exams and life**
- Use of ChatGPT is Encouraged for help and learning but **PLEASE DON'T COPY**
- Discussion with fellow Students is Encouraged for help and learning but **PLEASE DON'T COPY**
- **So please don't copy from anyone or anywhere**

No EXTENSION IN SUBMISSION DEADLINE

Submission of Both Parts Assignment by the Part One deadline on the portal will earn 25 % bonus Marks (Bonus Marks can be redeemed for future Marks Deficiencies)

PORTAL SUBMISSION INSTRUCTIONS

- **For each task**, please Submit **one single code file** named "t#.cpp" (must use the following provided template and the provided Classes), So for task 1 the file name will be t1.cpp and so on, so for complete assignment you will need to submit 4 .cpp files and on set of all the needed .h files for each of the task to run
- **Please submit the header files used to compile these .cpp files, otherwise the code will not compile**
- **Write Solution in English in Comments in each .cpp File (See example template below on how to describe the solution in English). The C++ code has no marks without this English Solution Description.** The English Solution must have have formal itemized formal steps (See example in template below)
- **Zip or RAR files have ZERO Marks**
- **Submissions by email will not be considered and graded**



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)
FACULTY OF INFORMATION TECHNOLOGY

MARKING SCHEME

- **No English Description** (see template below for an example) of solution **No marks** as it will be assumed that the **code is not your own**
- Code **does not compile or run** 50 % deducted (make sure to submit all needed header files)

Example Template File for Task 1:

```
// SPRING 2026 DATA STRUCTURES SECTION D20 ASSIGNMENT FIVE
// Name of Student:
// Registration of Student:
// File t1.cpp contains complete code of task one, that will compile and run
// Must use the following includes only, any other include must be justified in comments or will
disqualify the solution
#include <iostream>
using namespace std;
#include "Stack.h"
#include "MyStack.h"
#include "Queue.h"
#include "MyQueue.h"
/***** TASK DESCRIPTION *****/
// Copy task # (e.g. 1) description here (see the example below)
/*
    Read 5 integers from the user, calculate and display average and maximum value. You are
    allowed to use Stack ADT only and Queue only and no other ADT
*/
/***** TASK SOLUTION DESCRIPTION *****/
// Write the Steps of Solution here (follow the same format and style of the example below)
/*
Stack Only Solution without functions
-----
    1. Create a Stack of S Size 5    ----
                                     ----
    2. read values from the user and push on Stack until Stack is Full

        23
        ----
        29
        ----
        -23
        ----
        12
        ----
        -100
        ----

Stack: currentIndex = 4 and count = 5
```



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

3. Use Copy Constructor to make a Copy of Stack to preserve. Copy Stack CS = S
4. create 2 variables sum and max and initialize with CS.pop()
5. Until CS is empty pop from CS and save in a variable value
 6. Add value to sum
 7. compare values with max and save the largest in max
 8. increment a counter to get the number of values
8. display max and sum/counter;

Stack Only Solution functions

Function to Use:

- 1) Read
- 2) Stats

1. Create a Stack of S Size 5 -----

2. Call function read

23

29

-23

12

-100

Stack: currentIndex = 4 and count = 5

3. Use Copy Constructor to make a Copy of Stack to preserve. Copy Stack CS = S
4. create 2 variables sum and max and initialize with CS.pop()
5. Call function Stats

Queue Only Solution without any functions

1. Create a Queue of Q Size
2. read values from the user and enqueue in Queue until Queue is Full
Q: 22 → 345 → -55 → 100 → 234
3. Use Copy Constructor to make a Copy of Queue to preserve. Copy Queue CQ = Q
CQ: 22 → 345 → -55 → 100 → 234
4. create 2 variables sum and max and initialize with CQ.dequeue()
5. Until CQ is empty dequeue from CQ and save in a variable value
 6. Add value to sum
 7. compare values with max and save the largest in max



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)
FACULTY OF INFORMATION TECHNOLOGY

8. increment a counter to get the number of values
8. display max and sum/counter;

Queue Only Solution functions

Function to Use:

- 1) Read
- 2) Stats

1. Create a Queue of Q Size
2. Call Read
Q: 22 → 345 → -55 → 100 → 234
3. Use Copy Constructor to make a Copy of Queue to preserve. Copy Queue CQ = Q
CQ: 22 → 345 → -55 → 100 → 234
4. create 2 variables sum and max and initialize with CQ.dequeue()
5. Call Stats

*/

/****** GLOBAL FUNCTION PROTOTYPES *****/
// Write PROTOTYPES of All Global Functions Here

/****** Driver Function Main *****/
// CODE OF main here

/****** GLOBAL FUNCTION CODE *****/
// Write Codes of All Global Functions Here

There should be only 4 .CCP files one for each task along with the needed header files.
Any other submitted file will not be graded

ASSIGNMENT SIX Part One [50 Marks]

Due on Portal Only: **WED 11:59 PM June 17, 2026**

Task 1: AVLTree Implementation

Please write C+ code of all the public functions of the AVLTree (Class declaration AVLTree.h is provided below **(Must not modify)**), and use a suitable main to test these functions: **(All functions must be Iterative even any Helper function that you may use).**

```
// Node.h
template <class T>
class Node
```



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

```
{
public:
    T data;
    Node* left; // left child
    Node* right; // right child
    Node(T value)
    {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};
// AVLTree.h
template <class T>
class AVLTree
{
public:
    AVLTree();
    AVLTree(const AVLTree & avl);
    const AVLTree & operator = (const AVLTree & );
    ~AVLTree();
    void insert(const T & value);
    void remove(const T & value) ;
protected:
    Node * root;
};
```

You must use the following helper function as no Function should have more than 10 lines of code, otherwise it is assumed that the Code is not yours.

1. `int height(Node * root);`
2. `int balance(Node * root);`
3. `int balanceType(Node * parent, T newValue);` // returns the type of imbalance of parent after insertion of a new value at one the children of parent: returns 0 for a balanced tree; 1 for left-left imbalance, 2 for right-right imbalance, 3 for left-right imbalance and 4 for right-left imbalance.
4. `Node * leftRotate(Node * root);` // rotate left from child of root
5. `Node * rightRotate(Node * root);` // rotate right from child of root
6. `Node * bstInsert(Node * root, T value);` // insert in a BinarySearchTree and return the Node where the new Node is inserted
7. `Node * insert(Node * root, T value);` // insert a new value in the AVLTree; **must use above helper function as appropriate**
8. `Node * bstDelete(Node * n, T value);` // delete value from a BinarySearchTree and returns the replacement Node after deletion



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)
FACULTY OF INFORMATION TECHNOLOGY

- Node * delete(Node * root, T value); // delete value from the AVLTree and returns replacement Node after deletion. ; **must use above helper function as appropriate**

Task 2: MaxHeap Implementation

Please write C+ code of all the public functions of **the MinHeap** (Class declaration AVLTree.h is provided below. (**Must not modify**), and use a suitable main to test these functions:(**All functions must be iterative even any Helper function that you may use**).

```
// MaxHeap.h
template <class T>
class MaxHeap
{
public:
    // Constructor
    MaxHeap(int capacity);
    MaxHeap(const MaxHeap & mh);
    const MaxHeap & operator = (const MaxHeap & );
    ~MaxHeap(){}
    void insert(const T & value);
    void remove(const T & value);
    bool search(const T & value) const ;
    void heapSort();
    bool isEmpty() const;
    bool isFull() const;
    void display() const;
protected:
    T *harr; // pointer to array of elements in heap
    int capacity; // maximum possible size of min heap
    int currentSize; // Current number of elements in min heap
    // to heapify a subtree with the root at given index
    void MaxHeapify(int i);
};
```

You must use iterative helper function as no Function should have more than 10 lines of code, otherwise it is assumed that the Code is not yours.

ASSIGNMENT SIX Part Two [50 Marks]

Due on Portal Only: **WED 11:59 PM June 22, 2026**

Task 3: BinarySerachTree Application

Main application of the BinarySearchTree is to store values for quick search. In this task, you will write a complete program that does the following:

- Read text from a text file and stores the words in a BinarySearchTree. (when adding to the tree, lowercase and UPPERCASE character are treated as different



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

2. Write a BinarySearchTree of words to a text file
3. Able to search a word from the BinarySearchTree
4. Able to delete a word in the BinarySearchTree
5. Able to insert a new word in the BinarySearchTree
6. Able to process the words in the BinarySearchTree: Convert lowercase characters to UPPERCASE and vice-versa (after the conversion any duplicated words must be removed)

Task 4: MaxHeap and AVLTree Application

In this task you will implement an user ID-Password mechanism for an application that stores sorted user IDs (strings) in a MaxHeap named ID and ID-Passwords pairs in an AVLTree password. An ID-Password pair is basically a size 2 array of strings. The ID-Password pair are inserted in the AVLTree password based on ID.

For example: (user123-password123) is stored as {"user123", "password123"} and this pair will be stored as an AVLTree Node

You will need to the following:

1. Use MaxHeap class and create an instance ID
2. Use AVLTree and create an instance password
3. Provide a menu based main that provides the following functionality:
 - a. Add a new user ID and password
 - b. Remove a user by user ID (ID is removed from MaxHeap ID as well as the Node from AVLTree password)
 - c. Search password using a user ID

Display User ID and Passwords in a Table form

Task 5: Hashtable Implementation and Application Tree

1. Implement all the public functions of the following Hashtable.h (**mustn't modify**) Class

```
template <class T>
class Hashtable
{
public:
    Hashtable(int capacity);
    Hashtable(const Hashtable &);
    const Hashtable & operator = (const Hashtable & );
    ~Hashtable();
    void insert(T value);
    void remove(T value);
    T search(T key);
    void display();
protected:
    T * values;
    int capacity;
```



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

```
int count; // count of values stored
int getHashCode(int value)
{
    return value % capacity;
}
int getHashCode(char * s)
{
    int code = 0;
    for(int i = 0; i > strlen(s); i++)
    {
        code += s[i];
    }
    return code % strlen;
}
};
```

For collisions use chaining using an AVLTree, so each value mapping to the same index is stored in an AVLTree.

2. Implement word-level Hoffman Coding using the Hashtable class as following and provide main function to test your implementation:
 - a. Read text from file "text.txt", calculate the hashcode of each string and store each string in a Hashtable, named text, using chaining (use AVLTree)
 - b. Create another Hashtable, named strings and store the single copies of Each string using the already calculated hashcode. So the new Hashtable will not have duplicates.
 - c. Calculate frequency of each word and save the frequencies in another hashtable, named frequencies, using the hashcodes of the corresponding string
 - d. Now assign a binary code to each string with the most frequent string with the lowest binary number i.e. 0 and store these codes in another hashtable, named codes, using the hashcodes of the corresponding string
 - e. For Hoffman Compression, replace each string in hashtable text with corresponding binary code
 - f. For Decompression, replace each code in hashtable text with corresponding string